ShadowShare

BACKEND SPECIFICATION · V2.0

The blind vault.

A complete specification for the Python/Flask service that stores opaque ciphertext, enforces TTL, and exposes the rate-limited HTTPS API — without ever knowing what it holds.

FRAMEWORK Flask 3.0	LANGUAGE Python 3.11+
WSGI Gunicorn × 4	DATABASE PostgreSQL 15
OBJECT STORAGE DigitalOcean Spaces (S3)	REVERSE PROXY Nginx 1.24

Stateless · No Plaintext · Rate Limited · TTL-Enforced

TABLE OF CONTENTS

1. Backend Mission & Scope §1
2. Technology Stack §2
3. Service Architecture §3
4. Project Layout §4
5. Complete API Contracts §5
6. Database Schema & Migrations §6
7. Object Storage Layer §7
8. Chunked Upload Pipeline (F10) §8
9. Burn-After-Read Mechanism (F1) §9
10. Sender Tokens (F3, F4) §10
11. Background Jobs §11
12. Rate Limiting & Abuse Mitigation §12
13. Security Headers, CORS, CSP §13
14. Logging, Metrics, Observability §14
15. Configuration & Environment §15
16. Deployment Topology §16
17. CI/CD Pipeline §17
18. Disaster Recovery & Backups §18
19. Backend Testing Plan §19
20. Backend Tickets §20

01 Backend Mission & Scope

The ShadowShare backend has exactly four responsibilities and nothing else:

1. **Accept opaque blobs** uploaded by the client and persist them durably to object storage.
2. **Stream opaque blobs** back to clients holding a valid `file_id`.
3. **Enforce ephemerality** — TTL sweep, burn-after-read deletion, sender-initiated kill switch.
4. **Resist abuse** — rate limiting, body-size caps, TLS, CORS, security headers.

It is engineered so that someone with full administrator access to the server, the database, and the object store still cannot reconstruct a single user file. This is achieved by storing only ciphertext + opaque metadata; the absence of keys, passwords, and filenames is enforced by the database schema itself.

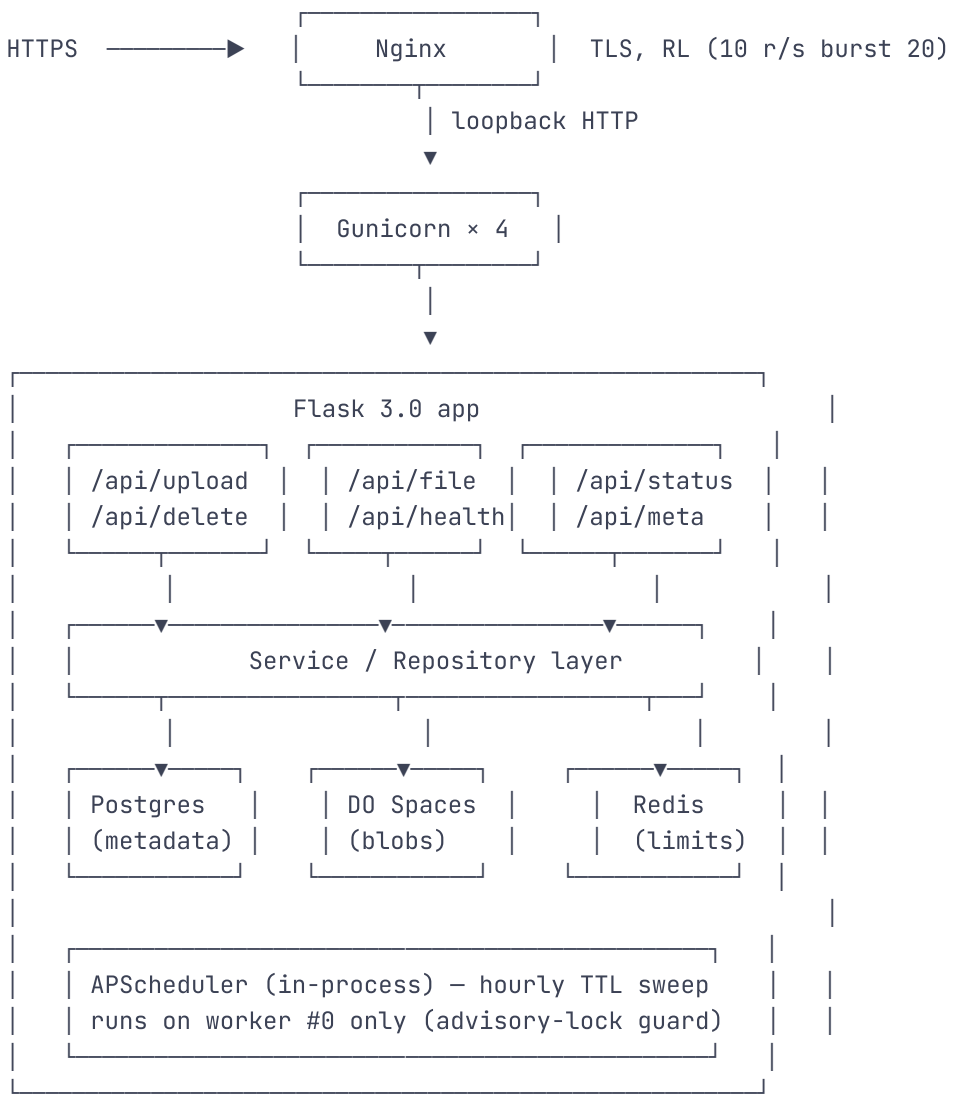
> **THE BLIND VAULT RULE**
>
> Every backend change must answer the question: *"After this change, can the server still be subpoenaed for nothing useful?"* If the answer becomes "no", the change is rejected.

02 Technology Stack

LAYER	TECH	VERSION	ROLE
Language	Python	3.11+	Mature stdlib; no FFI dependencies
Web Framework	Flask	3.0	Small, auditable surface
WSGI	Gunicorn	22.0	4 workers, sync mode
Reverse Proxy	Nginx	1.24	TLS termination, static assets, rate limiting
Database	PostgreSQL	15	Strict typing, TIMESTAMPTZ
DB Driver	psycopg	3.x	Modern Postgres driver
Migrations	Alembic	1.13	Versioned schema changes
Object Storage	DigitalOcean Spaces	—	S3-compatible; private bucket
S3 Client	boto3	1.34	Standard, audited
Rate Limiter	Flask-Limiter	3.5	Redis-backed counters
Cache / Counter Store	Redis	7	Distributed rate limit state
Background Jobs	APScheduler	3.10	Hourly TTL sweep, session GC
Validation	pydantic	2.x	Typed request/response models
Logging	structlog	24.x	JSON structured logs
Metrics	prometheus-flask-exporter	0.23	RED metrics

LAYER	TECH	VERSION	ROLE
Error tracking	Sentry	SDK 2.x	PII-scrubbed
TLS	Let's Encrypt + Certbot	—	Auto-renewal
Testing	pytest, testcontainers	8.x	Real Postgres + LocalStack S3

03 Service Architecture



The service is fully stateless. Any worker can serve any request. The only piece of long-lived in-process state is the APScheduler instance, which is bound to a Postgres advisory lock so it runs on exactly one worker.

04 Project Layout

```
shadowshare-api/
├── app/
│   ├── __init__.py             # Flask app factory
│   ├── config.py               # Pydantic Settings (env-driven)
│   ├── extensions.py           # db, limiter, scheduler, sentry init
│   ├── api/
│   │   ├── __init__.py
│   │   ├── upload.py           # upload init/chunk/status/complete
│   │   ├── file.py             # GET blob, GET meta
│   │   ├── delete.py           # DELETE (kill switch)
│   │   ├── status.py           # GET sender status
│   │   └── health.py
│   ├── models/
│   │   ├── file.py             # SQLAlchemy or psycopg row classes
│   │   └── session.py
│   ├── services/
│   │   ├── upload_service.py   # chunked upload state machine
│   │   ├── file_service.py     # download, burn handling
│   │   ├── delete_service.py
│   │   ├── status_service.py
│   │   └── storage.py          # S3 abstraction
│   ├── jobs/
│   │   ├── ttl_sweep.py
│   │   └── session_gc.py
│   ├── security/
│   │   ├── headers.py          # security headers middleware
│   │   ├── cors.py
│   │   ├── tokens.py           # delete_token verification
│   │   └── log_filters.py      # fragment/PII scrubbing
│   └── schemas/                # pydantic request/response models
│       ├── upload.py
│       ├── status.py
│       └── errors.py
├── migrations/                 # Alembic
│   └── versions/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── load/
├── deploy/
│   ├── nginx.conf
│   ├── shadowshare.service     # systemd unit
│   └── gunicorn.conf.py
├── pyproject.toml
```

```
├── requirements.txt
└── README.md
```

05 Complete API Contracts

All endpoints respond with JSON unless otherwise noted, use camelCase for response fields, and follow RFC 7807 problem details for errors. Authentication: none, by design — possession of `file_id` + `delete_token` is the entire credential model.

5.1 POST /api/upload/init

`POST` `/api/upload/init`

Purpose: Open a chunked upload session.

REQUEST

```
{
  "totalSizeBytes": 73654321,
  "totalChunks": 15,
  "settings": {
    "ttl": "7d",                      // one of: 1h | 24h | 7d | 30d
    "burnAfterRead": false,
    "hasAccessPassword": true,
    "maxDownloads": null              // optional positive int
  }
}
```

RESPONSE 201 CREATED

```
{
  "sessionId": "9a4f9c1b-...",
  "chunkSizeBytes": 5242880,
  "expiresAt": "2026-05-29T12:00:00Z"  // session expiry (24h)
}
```

ERRORS

`413 Payload Too Large`	totalSizeBytes > MAX_UPLOAD_BYTES
`422 Unprocessable Entity`	TTL not in ALLOWED_TTLS, invalid settings
`429 Too Many Requests`	Rate limit exceeded (10/h/IP)

5.2 PUT /api/upload/chunk/{sessionId}/{index}

`PUT` `/api/upload/chunk/{sessionId}/{index}`

Purpose: Upload one chunk of opaque ciphertext.

REQUEST

- **Headers:** `Content-Type: application/octet-stream`, `Content-Length`
- **Body:** raw bytes (≤ chunk size; last chunk may be smaller)

RESPONSE 204 NO CONTENT

Empty body. Server has persisted the chunk to the scratch area in Spaces and added `index` to the session's `received_chunks` array.

ERRORS

`404 Not Found`	session does not exist or has expired
`409 Conflict`	chunk index already received (idempotent — server still returns 204)
`413 Payload Too Large`	chunk exceeds chunk_size_bytes
`416 Range Not Satisfiable`	index out of bounds for totalChunks

5.3 GET /api/upload/status/{sessionId}

`GET` `/api/upload/status/{sessionId}`

RESPONSE 200 OK

```
{
  "sessionId": "9a4f9c1b-...",
  "totalChunks": 15,
  "receivedChunks": [0, 1, 2, 3, 4, 5, 6, 7],
  "nextChunkIndex": 8,
  "expiresAt": "2026-05-29T12:00:00Z"
}
```

5.4 POST /api/upload/complete/{sessionId}

`POST` `/api/upload/complete/{sessionId}`

Purpose: Reassemble chunks into the final blob, persist final row, return file identifiers + delete token.

REQUEST

```
{
  "encryptedMetadata": "<base64 bytes>",
  "encryptionIv": "<base64 bytes, 12 bytes>",
  "accessVerifier": {
    "verifier": "<base64 bytes>",
    "salt": "<base64 bytes>",
    "iv": "<base64 bytes>"
  }                                              // null if no access password
}
```

RESPONSE 201 CREATED

```
{
  "fileId": "8c4d1f2a-...",
  "deleteToken": "Yz9...",          // 256-bit base64url; returned ONCE
  "expiresAt": "2026-06-04T14:32:11Z",
  "blobSizeBytes": 73654321
}
```

ERRORS

`409 Conflict`	not all chunks received
`422 Unprocessable Entity`	invalid metadata payload

5.5 GET /api/file/{fileId}

`GET` `/api/file/{fileId}`

Purpose: Stream the opaque ciphertext blob.

RESPONSE 200 OK

- **Headers:** `Content-Type: application/octet-stream`, `Content-Length`, `X-Blob-Size`, `Cache-Control: no-store`
- **Body:** raw ciphertext bytes streamed from Spaces

SIDE EFFECTS

- `download_count += 1` on successful stream completion.
- `first_accessed_at = COALESCE(first_accessed_at, NOW())`.
- If `burn_after_read = true`: server registers a `call_on_close` hook that deletes the blob and the row atomically once the response is fully flushed.

ERRORS

`404 Not Found`	file_id does not exist
`410 Gone`	file expired, deleted, or burned
`429 Too Many Requests`	60/minute/IP limit

5.6 GET /api/file/{fileId}/meta

`GET` `/api/file/{fileId}/meta`

Purpose: Fetch the small encrypted metadata + public flags so the viewer can render the correct UI before downloading the full blob.

RESPONSE 200 OK

```
{
  "fileId": "8c4d1f2a-...",
  "blobSizeBytes": 73654321,
  "expiresAt": "2026-06-04T14:32:11Z",
  "burnAfterRead": false,
  "hasAccessPassword": true,
  "encryptedMetadata": "<base64>",
  "encryptionIv": "<base64>",
  "accessVerifier": {
    "verifier": "<base64>",
    "salt": "<base64>",
    "iv": "<base64>"
  }
}
```

5.7 DELETE /api/file/{fileId}

`DELETE` `/api/file/{fileId}?token={deleteToken}`

Purpose: Sender kill switch (F3).

SERVER LOGIC

```
computed = sha256(token).digest()
if not hmac.compare_digest(row.delete_token_hash, computed):
    return 403
delete blob from Spaces
delete row from Postgres
return 204
```

ERRORS

`403 Forbidden`	token mismatch
`404 Not Found`	file_id does not exist
`410 Gone`	already deleted / expired

5.8 GET /api/status/{fileId}

`GET` `/api/status/{fileId}?token={deleteToken}`

Purpose: Sender status (F4). The `deleteToken` doubles as the read-status capability — anyone who has the delete token by definition is the sender.

RESPONSE 200 OK

```
{
  "fileId": "8c4d1f2a-...",
  "status": "active",                    // active | expired | deleted
  "createdAt": "2026-05-28T14:32:11Z",
  "expiresAt": "2026-06-04T14:32:11Z",
  "downloadCount": 2,
  "firstAccessedAt": "2026-05-28T15:11:03Z",
  "burnAfterRead": false,
  "hasAccessPassword": true
}
```

> PRIVACY GUARANTEE ON STATUS
>
> This endpoint **never** returns recipient identifying data — no IPs, user agents, geolocations, referrers. None of those are stored in the first place. The fields above are the complete schema.

5.9 GET /api/health

`GET` `/api/health`

RESPONSE 200 OK

```
{ "status": "ok", "version": "2.0.0", "uptime_seconds": 4128 }
```

5.10 Standard Error Envelope (RFC 7807)

```
{
  "type": "https://shadowshare.io/errors/rate-limit",
  "title": "Too Many Requests",
  "status": 429,
  "detail": "Upload limit of 10 per hour exceeded.",
  "retryAfter": 1342
}
```

06 Database Schema & Migrations

6.1 Initial Migration

```
-- migrations/versions/0001_initial.sql

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE files (
    file_id              UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    blob_path            VARCHAR(500) NOT NULL UNIQUE,
    blob_size_bytes      BIGINT       NOT NULL,
    created_at           TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    expires_at           TIMESTAMPTZ  NOT NULL,
    download_count       INTEGER      NOT NULL DEFAULT 0,
    first_accessed_at    TIMESTAMPTZ  NULL,
    burn_after_read      BOOLEAN      NOT NULL DEFAULT FALSE,
    burned               BOOLEAN      NOT NULL DEFAULT FALSE,
    has_access_password  BOOLEAN      NOT NULL DEFAULT FALSE,
    access_verifier      BYTEA        NULL,
    access_verifier_salt BYTEA        NULL,
    access_verifier_iv   BYTEA        NULL,
    encrypted_metadata   BYTEA        NOT NULL,
    encryption_iv        BYTEA        NOT NULL,
    delete_token_hash    BYTEA        NOT NULL,
    max_downloads        INTEGER      NULL,
    CHECK (blob_size_bytes > 0 AND blob_size_bytes ⩽ 1073741824),
    CHECK (octet_length(encryption_iv) = 12),
    CHECK (octet_length(delete_token_hash) = 32)
);
CREATE INDEX idx_files_expires_at ON files (expires_at);
CREATE INDEX idx_files_burned     ON files (burned) WHERE burned = TRUE;

CREATE TABLE upload_sessions (
    session_id           UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    total_size_bytes     BIGINT       NOT NULL,
    total_chunks         INTEGER      NOT NULL,
    chunk_size_bytes     INTEGER      NOT NULL DEFAULT 5242880,
    received_chunks      INTEGER[]    NOT NULL DEFAULT '{}',
    scratch_prefix       VARCHAR(500) NOT NULL,    -- Spaces key prefix
    settings             JSONB        NOT NULL,
    created_at           TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    expires_at           TIMESTAMPTZ  NOT NULL DEFAULT NOW() + INTERVAL '24 hours',
    completed_at         TIMESTAMPTZ  NULL
```

```
);
CREATE INDEX idx_sessions_expires ON upload_sessions (expires_at);
```

6.2 What is intentionally absent

ABSENT COLUMN	WHY
`filename`	Stored inside `encrypted_metadata` (opaque ciphertext)
`content_type`	Same as above
`uploader_ip`	Never logged or persisted
`recipient_ip`	Never logged or persisted
`password / password_hash`	Server never sees password
`encryption_key`	Lives only in URL fragment, in client memory
`kdf_salt`	Lives in URL fragment
`delete_token (plaintext)`	Only SHA-256 hash persisted

6.3 Least-Privilege DB User

```
CREATE USER shadowshare_app WITH PASSWORD '...';
GRANT CONNECT ON DATABASE shadowshare TO shadowshare_app;
GRANT USAGE ON SCHEMA public TO shadowshare_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON files, upload_sessions TO shadowshare_app;
-- No DDL, no superuser, no createdb.
```

07 Object Storage Layer

7.1 Bucket Configuration

- **Region:** `nyc3` (or chosen region).
- **Visibility:** Private. No public-read ACL anywhere.
- **Access:** Single IAM/Spaces key pair scoped to this bucket only.
- **CORS:** No browser direct access to Spaces — clients only ever talk to the Flask API. The API streams from Spaces.
- **Lifecycle:** A safety-net 35-day lifecycle rule auto-deletes any orphan object (the application-level TTL sweep should always beat this).

7.2 Key Naming Convention

```
blobs/{yyyy}/{mm}/{dd}/{file_id}.bin           # final blobs
sessions/{session_id}/chunks/{index}.part       # scratch chunks during chunked upload
```

7.3 Storage Service Interface

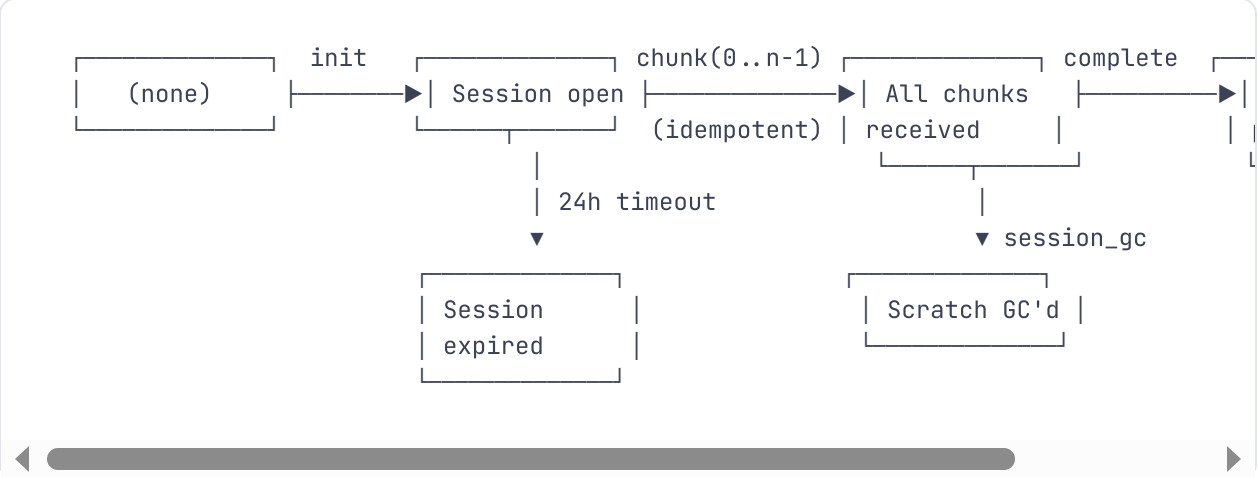
```
# app/services/storage.py
class BlobStore:
    def put_chunk(self, session_id: str, index: int, data: bytes) → None
    def assemble(self, session_id: str, total_chunks: int, file_id: str) → str  # returns b
    def stream_blob(self, blob_path: str) → Iterator[bytes]
    def delete_blob(self, blob_path: str) → None
    def delete_session_scratch(self, session_id: str) → None
```

7.4 Assemble Operation

For files up to 100 MB v1, `assemble` streams each chunk in order and uploads the concatenation as a single PutObject. For larger files (v2.1 roadmap, up to 1 GB), it switches to S3 multipart upload, treating each scratch chunk as one upload part.

08 Chunked Upload Pipeline (F10)

8.1 State Machine



8.2 Concurrency & Idempotency

- Chunk uploads use `UPDATE upload_sessions SET received_chunks = array_append(received_chunks, $1) WHERE NOT $1 = ANY(received_chunks)` for atomic dedupe.
- The complete endpoint takes a row-level lock: `SELECT … FROM upload_sessions WHERE session_id = $1 FOR UPDATE`.
- Out-of-order chunks are accepted; only the final reassembly requires all indices present.

8.3 Resume Logic

`GET /api/upload/status/{sessionId}` returns the array; the client computes `nextChunkIndex` as the lowest missing integer in `[0, totalChunks)` and resumes from there.

09 Burn-After-Read Mechanism (F1)

The challenge: deletion must happen *only* after the byte stream completes successfully, not when the response headers go out.

9.1 Implementation

```
@bp.get('/api/file/<uuid:file_id>')
def get_file(file_id):
    row = file_service.load_for_read(file_id)
    if row is None: abort(404)
    if row.burned or row.expires_at < utcnow(): abort(410)

    def stream():
        bytes_sent = 0
        for chunk in storage.stream_blob(row.blob_path):
            bytes_sent += len(chunk)
            yield chunk
        # Generator exhausted ⇒ stream completed without exception
        file_service.on_download_complete(row, bytes_sent)

    response = Response(stream(), mimetype='application/octet-stream')
    response.headers['Content-Length'] = str(row.blob_size_bytes)
    response.headers['X-Blob-Size']   = str(row.blob_size_bytes)
    response.headers['Cache-Control'] = 'no-store'
    return response

# file_service.on_download_complete:
def on_download_complete(row, bytes_sent):
    if bytes_sent ≠ row.blob_size_bytes:
        return                       # client aborted; do NOT burn
    with db.transaction():
        db.execute("UPDATE files SET download_count = download_count + 1, "
                   "first_accessed_at = COALESCE(first_accessed_at, NOW()) "
                   "WHERE file_id = %s", [row.file_id])
        if row.burn_after_read:
            db.execute("UPDATE files SET burned = TRUE WHERE file_id = %s", [row.file_id])
    if row.burn_after_read:
        storage.delete_blob(row.blob_path)
        with db.transaction():
            db.execute("DELETE FROM files WHERE file_id = %s", [row.file_id])
```

9.2 Race protection

Two concurrent downloads racing on a burn-after-read file are serialised by a per-row Postgres advisory lock:

```
SELECT pg_advisory_xact_lock(hashtext(file_id::text));
```

The losing transaction observes `burned = TRUE` and returns 410.

10 Sender Tokens (F3, F4)

10.1 Token Generation

```
# app/security/tokens.py
import secrets, hashlib

def issue_delete_token() → tuple[str, bytes]:
    raw = secrets.token_bytes(32)              # 256-bit
    encoded = base64.urlsafe_b64encode(raw).rstrip(b'=').decode()
    return encoded, hashlib.sha256(raw).digest()
```

10.2 Token Verification

```
def verify_delete_token(encoded: str, stored_hash: bytes) → bool:
    try:
        raw = base64.urlsafe_b64decode(encoded + '==')
    except Exception:
        return False
    computed = hashlib.sha256(raw).digest()
    return hmac.compare_digest(stored_hash, computed)   # constant-time
```

10.3 Token Reuse

The same `delete_token` grants two capabilities: revoke (DELETE) and status (GET /api/status). This is intentional and equivalent in trust scope: anyone holding the token is the sender.

11 Background Jobs

11.1 Hourly TTL Sweep

```
# app/jobs/ttl_sweep.py
def run():
    with db.session() as s:
        # Acquire a Postgres advisory lock so only one worker runs the sweep
        got = s.execute("SELECT pg_try_advisory_lock(742019)").scalar()
        if not got: return
        try:
            rows = s.execute(
                "SELECT file_id, blob_path FROM files "
                "WHERE expires_at < NOW() OR burned = TRUE LIMIT 500"
            ).fetchall()
            for r in rows:
                storage.delete_blob(r.blob_path)
                s.execute("DELETE FROM files WHERE file_id = %s", [r.file_id])
            s.commit()
        finally:
            s.execute("SELECT pg_advisory_unlock(742019)")
```

11.2 Daily Session GC

Deletes upload sessions older than 24 hours along with their scratch chunks in Spaces. Same advisory-lock pattern.

11.3 Scheduler Configuration

```
# app/extensions.py
scheduler = BackgroundScheduler(timezone='UTC')
scheduler.add_job(ttl_sweep.run,    'interval', hours=1, id='ttl_sweep',  max_instances=1)
scheduler.add_job(session_gc.run,   'interval', hours=6, id='session_gc', max_instances=1)
scheduler.start()
```

12 Rate Limiting & Abuse Mitigation

12.1 Limits

ENDPOINT	LIMIT	SCOPE
POST /api/upload/init	10 / hour	per IP
PUT /api/upload/chunk/*	500 / hour	per IP (covers a 100MB file at 5MB chunks × many retries)
POST /api/upload/complete/*	20 / hour	per IP
GET /api/file/{id}	60 / minute	per IP
GET /api/file/{id}/meta	120 / minute	per IP
DELETE /api/file/{id}	30 / minute	per IP
GET /api/status/{id}	120 / minute	per IP

Counters live in Redis with sliding-window precision. When a limit is exceeded the server returns `429 Too Many Requests` with a `Retry-After` header.

12.2 Body Size Enforcement (three layers)

1. **Nginx:** `client_max_body_size 105m;` (room for 100 MB + a little overhead).
2. **Flask:** `MAX_CONTENT_LENGTH = 104_857_600`.
3. **DB:** `CHECK (blob_size_bytes ⩽ 1073741824)` — accepts up to 1 GB at schema level; the app config narrows this for v2.0.

12.3 Fail2ban

Jail `nginx-limit-req` watches the Nginx error log for rate-limit triggers and bans IPs producing > 50 rejections per 10 minutes for 1 hour.

12.4 Abuse takedown

Because the server cannot read content, the abuse-handling policy is published as: *(a)* we ban file IDs reported through the documented abuse channel, *(b)* we cannot inspect or filter content, and *(c)* the sender can revoke any file they uploaded with their delete token.

13 Security Headers, CORS, CSP

13.1 Headers (set by Nginx + reinforced by Flask middleware)

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inl
                         img-src 'self' data: blob:; font-src 'self'; connect-src 'self';
                         frame-ancestors 'none'; base-uri 'self'; form-action 'none';
                         upgrade-insecure-requests
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Permissions-Policy: geolocation=(), camera=(), microphone=(), payment=(), usb=(),
                    interest-cohort=(), browsing-topics=()
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Cache-Control: no-store    (on /api/*)
```

13.2 CORS

```
Access-Control-Allow-Origin: https://shadowshare.io
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type
Access-Control-Max-Age: 600
```

No wildcard. The single allowed origin is configured via `ALLOWED_ORIGIN` env var.

13.3 TLS Configuration (Nginx)

```
ssl_protocols TLSv1.3 TLSv1.2;
ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:...;
ssl_prefer_server_ciphers off;
ssl_session_cache shared:SSL:50m;
ssl_session_timeout 1d;
ssl_session_tickets off;
ssl_stapling on;
ssl_stapling_verify on;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload";
```

Target: **SSL Labs A+**. Reassessed weekly via CI.

14 Logging, Metrics, Observability

14.1 What we log

EVENT	FIELDS
Upload init	session_id, totalSizeBytes, ttl, chunkCount, request_id
Upload chunk	session_id, chunk_index, chunk_size, latency_ms
Upload complete	file_id, blob_size, request_id
Download	file_id, bytes_sent, status (complete/aborted), latency_ms
Delete (kill switch)	file_id, outcome
TTL sweep	rows_deleted, blobs_deleted, duration_ms
Rate limit	endpoint, ip_hash (truncated SHA-256), limit_name

14.2 What we never log

- Request bodies of any sort.
- The URL fragment of any URL (referer header is stripped at Nginx).
- Encrypted blob bytes (not even size in error scenarios where bytes might appear).
- Delete token values — only hashed values appear in any context.
- Raw client IP. We hash IPs with a daily-rotating salt for rate-limit forensics; the salt is shredded after 24 hours.

14.3 Log Filter

```
# app/security/log_filters.py
class FragmentStripperFilter(logging.Filter):
    URL_FRAGMENT = re.compile(r'#[^\s"']+')
    def filter(self, record):
        if isinstance(record.msg, str):
            record.msg = self.URL_FRAGMENT.sub('#<redacted>', record.msg)
        return True
```

14.4 Metrics (Prometheus)

- `shadowshare_uploads_total{result}`
- `shadowshare_downloads_total{result}`
- `shadowshare_blob_bytes_total{direction}`
- `shadowshare_ttl_sweep_rows_deleted_total`
- `shadowshare_rate_limit_triggered_total{endpoint}`
- `shadowshare_request_duration_seconds{endpoint,method,status}` (histogram)

14.5 Sentry

Configured with a `before_send` hook that drops events containing query strings or fragments. Request body capture is disabled.

15 Configuration & Environment

```
# Required
FLASK_ENV=production
SECRET_KEY=<64-byte hex from `openssl rand -hex 32`>
DATABASE_URL=postgresql://shadow:<pw>@127.0.0.1:5432/shadowshare
REDIS_URL=redis://127.0.0.1:6379/0
DO_SPACES_KEY=<...>
DO_SPACES_SECRET=<...>
DO_SPACES_BUCKET=shadowshare-blobs
DO_SPACES_ENDPOINT=https://nyc3.digitaloceanspaces.com
ALLOWED_ORIGIN=https://shadowshare.io

# Tunables
MAX_UPLOAD_BYTES=104857600         # 100 MB
CHUNK_SIZE_BYTES=5242880           # 5 MB
DEFAULT_TTL=7d
ALLOWED_TTLS=1h,24h,7d,30d
UPLOAD_SESSION_TTL_HOURS=24
TTL_SWEEP_INTERVAL_HOURS=1

# Optional
SENTRY_DSN=
LOG_LEVEL=INFO
PROMETHEUS_ENABLED=true
```

Loaded via Pydantic Settings — type-checked at startup; missing required vars cause an immediate fatal abort with a clear message.

16 Deployment Topology

16.1 Host

- DigitalOcean Droplet, Ubuntu 22.04 LTS, 4 vCPU / 8 GB RAM (v2.0 baseline).
- UFW firewall: 22, 80, 443 inbound only.
- SSH: key-only, root disabled, fail2ban on.
- Automatic security updates: `unattended-upgrades` enabled.

16.2 systemd Unit

```
# /etc/systemd/system/shadowshare.service
[Unit]
Description=ShadowShare API
After=network.target postgresql.service redis-server.service

[Service]
User=shadowshare
Group=shadowshare
WorkingDirectory=/opt/shadowshare/api
EnvironmentFile=/etc/shadowshare/env
ExecStart=/opt/shadowshare/.venv/bin/gunicorn \
          --workers 4 \
          --bind 127.0.0.1:8000 \
          --timeout 120 \
          --access-logfile - \
          app:app
Restart=on-failure
RestartSec=5
PrivateTmp=true
NoNewPrivileges=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/var/log/shadowshare
CapabilityBoundingSet=

[Install]
WantedBy=multi-user.target
```

16.3 Nginx Snippet

```
server {
    listen 443 ssl http2;
    server_name shadowshare.io;

    ssl_certificate     /etc/letsencrypt/live/shadowshare.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/shadowshare.io/privkey.pem;

    client_max_body_size 105m;

    # Static React build
    root /opt/shadowshare/web/dist;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
        proxy_request_buffering off;     # critical for chunked uploads
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
    }

    location / {
        try_files $uri /index.html;
    }
}

server { listen 80; server_name shadowshare.io; return 301 https://$host$request_uri; }
```

17 CI/CD Pipeline

17.1 GitHub Actions Workflow

```
.github/workflows/main.yml
```

```yaml
jobs:
  lint-test:
    runs-on: ubuntu-22.04
    services:
      postgres: postgres:15
      redis:    redis:7
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5  { python-version: '3.11' }
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: mypy app
      - run: pytest --cov=app --cov-fail-under=80
      - run: pip-audit

  build:
    needs: lint-test
    steps:
      - run: docker build -t shadowshare:${{ github.sha }} .
      - run: trivy image shadowshare:${{ github.sha }}

  deploy:
    if: github.ref == 'refs/heads/main'
    needs: build
    steps:
      - name: SSH deploy
        run: |
          ssh deploy@$HOST 'cd /opt/shadowshare && \
            git fetch && git checkout ${{ github.sha }} && \
            ./scripts/deploy.sh'
      - name: Smoke
        run: curl --fail https://shadowshare.io/api/health
```

17.2 Deploy Script

```
#!/bin/bash
set -euo pipefail
.venv/bin/pip install -r requirements.txt
.venv/bin/alembic upgrade head
sudo systemctl reload shadowshare
sleep 2
curl --fail http://127.0.0.1:8000/api/health
```

18 Disaster Recovery & Backups

ASSET	BACKUP STRATEGY	RTO	RPO
PostgreSQL (metadata)	DO automated daily snapshots + pg_basebackup hourly to encrypted off-droplet storage	1 hour	1 hour
DO Spaces (blobs)	No backup — blobs are ephemeral by design. Loss is acceptable.	n/a	n/a
Redis (rate-limit counters)	No backup — losing counters means temporarily relaxed limits; acceptable.	n/a	n/a
Application code	GitHub origin + signed tags	10 min	0
Secrets (env file)	Encrypted with `age`, stored in a separate private repo + sealed envelope offline	30 min	0

> **ASYMMETRIC DR POSTURE**
>
> Because all user file data is encrypted by the user and intended to be ephemeral, we choose not to back up blob storage — doing so would create a longer-lived attack surface for ciphertext we have promised to delete. Loss of the blob store inconveniences users (links return 410) but does not violate any guarantee.

19 Backend Testing Plan

19.1 Unit (pytest)

- `tests/unit/test_tokens.py` — token issuance, constant-time verification, malformed token rejection.
- `tests/unit/test_validation.py` — Pydantic schemas reject malformed bodies.
- `tests/unit/test_log_filter.py` — fragment stripper covers every URL shape.
- `tests/unit/test_rate_limiter.py` — limits trip at the right counts.

19.2 Integration (testcontainers)

Real Postgres + LocalStack S3 + Redis spun up per test session.

- Full upload → file row exists → blob exists → metadata is opaque bytes.
- Resume: drop chunk 3, status reports missing, re-upload succeeds, complete works.
- Burn-after-read: 1st download succeeds, row+blob gone, 2nd download → 410.
- Burn race: two concurrent downloads — exactly one succeeds, other returns 410.
- Kill switch: wrong token → 403, right token → 204 and row+blob gone.
- Status: returns correct counts and timestamps; never includes IPs or recipient identifiers.
- TTL sweep: row past expires_at is removed within 1 hour cycle.
- Rate limit: 11th upload init in an hour returns 429 with Retry-After.
- Tampered blob (bit flipped in storage) — backend serves it; the test then verifies frontend rejects.

19.3 Security Tests

- No endpoint returns IPs, user agents, referers, or filenames.
- CORS preflight from a non-allowed origin is rejected.
- Body larger than MAX_UPLOAD_BYTES returns 413 from Nginx (not from Flask — caught earlier).
- SSRF: storage endpoint is the only place that talks to Spaces; no user input flows into URL construction.

- SQL injection: parameterised queries everywhere; static analysis with bandit.

19.4 Load (k6)

```
scenarios:
  steady_state:
    executor: constant-arrival-rate
    rate: 50            # 50 req/s
    duration: 5m
    preAllocatedVUs: 50
thresholds:
  http_req_duration:    ['p(95)<120']    # ms
  http_req_failed:      ['rate<0.001']
```

20 Backend Tickets

Backend-owned tickets with acceptance criteria.

SS-001 Provision DigitalOcean droplet

Ubuntu 22.04 LTS, key-only SSH, UFW (22/80/443), fail2ban, unattended-upgrades.
AC: `nmap` shows only 22/80/443; root login refused.

SS-002 Nginx + Gunicorn + systemd

TLS termination, systemd unit with hardening flags, log rotation.
AC: `systemctl status shadowshare` healthy; SSL Labs A+.

SS-003 PostgreSQL 15 + initial schema

Alembic migration 0001 applied.
AC: `files` + `upload_sessions` exist; least-priv user works.

SS-004 DO Spaces bucket + TLS certs

Private bucket, Let's Encrypt + Certbot auto-renew, 35-day safety-net lifecycle.
AC: bucket lists 0 public-read objects; cert auto-renew dry-run passes.

SS-008 Single-shot upload endpoint

v1.0 baseline; superseded by SS-021 in v2.0 but retained for tests.

SS-009 GET /api/file streaming

Streams from Spaces; increments counters atomically.
AC: p95 latency ≤ 120 ms; aborted streams do NOT increment.

SS-010 Hourly TTL sweep

APScheduler + advisory lock.
AC: single-worker execution proven; sweep deletes both row and blob.

SS-014 Rate limiting + size enforcement

Flask-Limiter + Nginx body cap.
AC: 11th upload init in an hour → 429 with Retry-After.

SS-015 Security headers + CORS

All headers from §13.
AC: securityheaders.com grade A+; preflight from disallowed origin returns 403.

SS-019 Burn-after-read v2.0

F1. Atomic delete on stream completion; race-safe.
AC: two concurrent reads → exactly one 200, one 410.

SS-020-BE TTL validation v2.0

F2. Reject TTLs not in ALLOWED_TTLS.
AC: bad ttl → 422.

SS-021 Chunked upload pipeline v2.0

F10. init / chunk / status / complete; resumable.
AC: 100 MB upload with mid-flight kill of chunk 7 reassembles correctly.

SS-022 Delete token issuance + verification

v2.0

F3. 256-bit, base64url; SHA-256 hash stored; constant-time compare.
AC: wrong token → 403; right token → 204; row + blob gone.

SS-023-BE GET /api/status endpoint

v2.0

F4. Token-gated; returns ONLY non-PII fields.
AC: response schema matches §5.8 exactly; no IP appears in any field.

SS-024-BE Access verifier storage

v2.0

F5. Persist `access_verifier`, `access_verifier_salt`, `access_verifier_iv` as opaque bytes; expose flag in meta.
AC: server has no code path that decrypts the verifier.

SS-032 Redis rate limiter

v2.0

Switch Flask-Limiter storage from memory to Redis; counters survive worker restart.

SS-033-BE Encrypted metadata persistence

v2.0

Store `encrypted_metadata` bytes; reject any plaintext filename if client mistakenly sends one.

SS-038-BE k6 load profile in CI

v2.0

50 RPS for 5 minutes; p95 budget enforced.
AC: CI fails if p95 exceeds 120 ms.

SS-039-BE Structured logging + Sentry scrubbing

v2.0

structlog JSON; `FragmentStripperFilter`; Sentry `before_send` drops query strings.
AC: manual review of one hour of production logs shows no fragment, body, or filename.